Individual Project

Manya Jain 2024102068

Phase 1: Data Collection and Tokenizer Construction

In this project, I built and compared two small language models from scratch to see how different languages and dataset sizes affect training. I trained **Model H** for Hindi (which has a lot of data available) and **Model L** for Nepali (which has much less data).

To keep the comparison fair, the two models don't share any training text, vocabularies, or tokenizers. This way, any differences in how well they generate text or learn patterns come entirely from the languages themselves. I wrote everything directly in PyTorch including the web scrapers, tokenizers, attention layers, and training loops without using any pre-trained weights or Hugging Face libraries.

Corpus Construction. The foundation of this project began with collecting and curating large-scale monolingual corpora of comparable magnitude (∼500M words). Sourcing Hindi data was straightforward, I combined the Sangraha Indic corpus, Hindi Wikipedia, government press releases, movie subtitles, and news archives. In contrast, gathering Nepali at a similar scale posed a significant low-resource challenge. Academic corpora such as IRIIS were too small on their own. To bridge the gap, I wrote targeted scrapers to collect articles from Nepali national news outlets, online literary portals, and digital archives.

Data Cleaning. Web text contains encoding errors, boilerplate, and mixed-script debris. I built a unified preprocessing cleaning pipeline that normalized text to Unicode NFC, filtered out lines with low Devanagari character purity (removing English snippets), and performed exact line-level deduplication to eliminate repeated crawl artifacts.

Table 1: Corpus composition after filtering and deduplication

Corpus Metric	Hindi (Model H)	Nepali (Model L)
Raw downloaded sources, words	461.4M	514.6M
Post-cleaning words	455.9M	513.0M
Post-tokenization estimate	549.4M	618.7M
Manually scraped share	19.9%	22.3%
Training split words	410.4M	461.5M
Validation split words	22.8M	25.8M
Test split words	22.7M	25.7M

To ensure genuine data collection effort beyond downloading pre-packaged dumps, I tracked a substantial manually scraped component in both languages (19.9% in Hindi, 22.3% in Nepali). Rigorous deduplication also prevented crawled snippets from leaking across the evaluation splits.

Each language was partitioned into a deterministic 90/5/5 train, validation, and test split. Although both final corpora reached similar post-cleaning volumes, their underlying domain diversity differs fundamentally. Hindi represents a rich mixture of conversational, journalistic, and literary registers. Nepali is heavily concentrated in formal journalism and archival text. As seen in Phase 2, this qualitative difference in domain variance has a major impact on pretraining loss and generative diversity.

Tokenizer Training

Having prepared the cleaned corpora, I trained custom Byte-Pair Encoding (BPE) tokenizers from scratch for each language. Because standard BPE implementations attempt to load the entire text into RAM (which is impossible for 500M-word corpora on standard compute), I implemented a memory-efficient streaming BPE trainer. The algorithm processes text in streaming chunks, tracks character pair frequencies with a min-heap, prunes singleton words, and dynamically updates adjacent co-occurrence counts after

each merge.

Model H and Model L deliberately use two different BPE variants, chosen from the start of the project as a point of comparison:

- **Hindi (Byte-level BPE):** Text is first decomposed into raw UTF-8 bytes before merges are learned (the same approach used by GPT-2). Because every possible byte is representable, this variant can encode any input, including rare symbols or mixed script, with a 0.00% unknown (UNK) rate. This suits Hindi's larger, more diverse corpus (news, Wikipedia, subtitles), which is more likely to contain irregular characters despite the cleaning pipeline.
- **Nepali (Unicode character-level BPE):** Merges are learned directly over Devanagari Unicode codepoints rather than bytes. This keeps the vocabulary file human-readable in the native script and ties each token directly to a valid character, at the cost of a small UNK rate (0.00011%, or just 7 tokens) on characters missing from the training vocabulary.

Training both variants let me directly compare a byte-level and a character-level tokenizer under matched conditions (same script family, same general cleaning pipeline). The trade-off is clear: byte-level BPE guarantees zero UNK at the cost of an unreadable, byte-mapped vocabulary file, while character-level BPE keeps the vocabulary readable but sacrifices absolute coverage.

When preparing for full Transformer pretraining in Phase 2, GPU memory constraints dictated balancing embedding parameter overhead against model depth. At a 32K vocabulary, the embedding matrix alone would consume over 16M parameters (nearly two-thirds of our entire parameter budget). Consequently, both tokenizers were fixed to exactly 16,000 vocabulary size for the pretraining runs. This preserved a healthy fertility rate while keeping the embedding size balanced with the rest of the model.

Table 2: Tokenizer measurements on held-out text at exactly 16,000 vocabulary size

Measure	**Hindi**	**Nepali**
Tokenizer type	Byte-level BPE	Character-level BPE
Phase 1 vocabulary size	16,000	16,000
Average characters per token	4.22	5.38
Fertility, tokens per word	1.2052	1.2059
Unknown-token rate	0.00%	0.00011%

Nepali requires a slightly larger number of characters per token (5.38 characters/token vs. 4.22 in Hindi) to reach identical fertility (1.205 tokens/word), reflecting its complex suffixes and postpositions. Finally, the entire training split was pre-tokenized and serialized into flat binary arrays of 16-bit integers, memory-mapped via `np.memmap` for zero-latency, batch-shuffled streaming into the Phase 2 training loop.

Phase 2: Model Implementation and Pretraining

With memory-mapped token streams ready, I implemented both GPT-style decoder-only Transformer models from scratch in PyTorch. To understand every tensor transformation and gradient flow, I deliberately avoided high-level modules like `nn.Transformer`. I hand-coded multi-head self-attention, causal masking, learned positional embeddings, layer normalization, and the feed-forward sublayers.

I used a Pre-LayerNorm architecture because it keeps training more stable early on compared to older Post-LN designs. The causal mask prevents tokens from looking ahead at future words by setting the upper-triangular attention scores to negative infinity. Inside the feed-forward network, I used GELU activations and expanded the hidden size to 3 times the model dimension (1,536). I chose learned absolute positional embeddings, which constrains the maximum sequence length to 256 tokens but is highly efficient. I empirically verified my causal mask implementation by confirming that altering token $t+1$ produced zero change in the output logits at position t. To save parameters and stay under my $\sim$24M limit, I shared the weights between the input embeddings and the final output layer (weight tying).

Table 3: Pretraining architecture and hyperparameter configuration

Parameter	Hindi (Model H)	Nepali (Model L)
Vocabulary used for pretraining	16,000	16,000
Transformer layers	6	6
Hidden dimension (d_{model})	512	512
Attention heads	8	8
Head dimension (d_{head})	64	64
Feed-forward dimension (d_{ff})	1,536	1,536
Context length	512	512
Dropout rate	0.1	0.1
Total parameters	24.22M	24.22M
Total pretraining steps	61,036	61,036

Both models were trained using AdamW with a peak learning rate of 5×10^{-4}, 2,000 steps of linear warmup, and cosine decay down to 10%. I engineered a fully resumable checkpointing system that periodically serialized the complete training state, allowing training to recover from transient node interruptions. Both models completed their 61,036-step training schedule stably without divergence, producing two fully trained base language models ready for comparative evaluation.

Language-Model Results

Both pretraining runs exhibited smooth, steady drop in loss. To evaluate generalization, I computed test loss, perplexity (PPL $= e^{\text{loss}}$), and Bits Per Byte (BPB $= \frac{\text{loss} \cdot \log_2 e}{\text{bytes per token}}$). Evaluating bits per byte is essential here because raw cross-entropy is parameterized over tokens, making direct cross-lingual loss comparisons misleading when tokenizers differ in byte density.

Table 4: Held-out language-model evaluation metrics

Evaluation Metric	Hindi (Model H)	Nepali (Model L)
Held-out test loss (nats)	3.666	3.592
Perplexity (PPL)	39.11	36.32
Bits per byte (BPB)	0.513	0.394
Average bytes per token	10.31	13.16
Final validation loss (nats)	3.649	3.605

Nepali achieved lower test loss (3.592 vs. 3.666), lower perplexity (36.32 vs. 39.11), and noticeably lower bits per byte (0.394 vs. 0.513 BPB) than Hindi. However, interpreting this as evidence that Nepali is

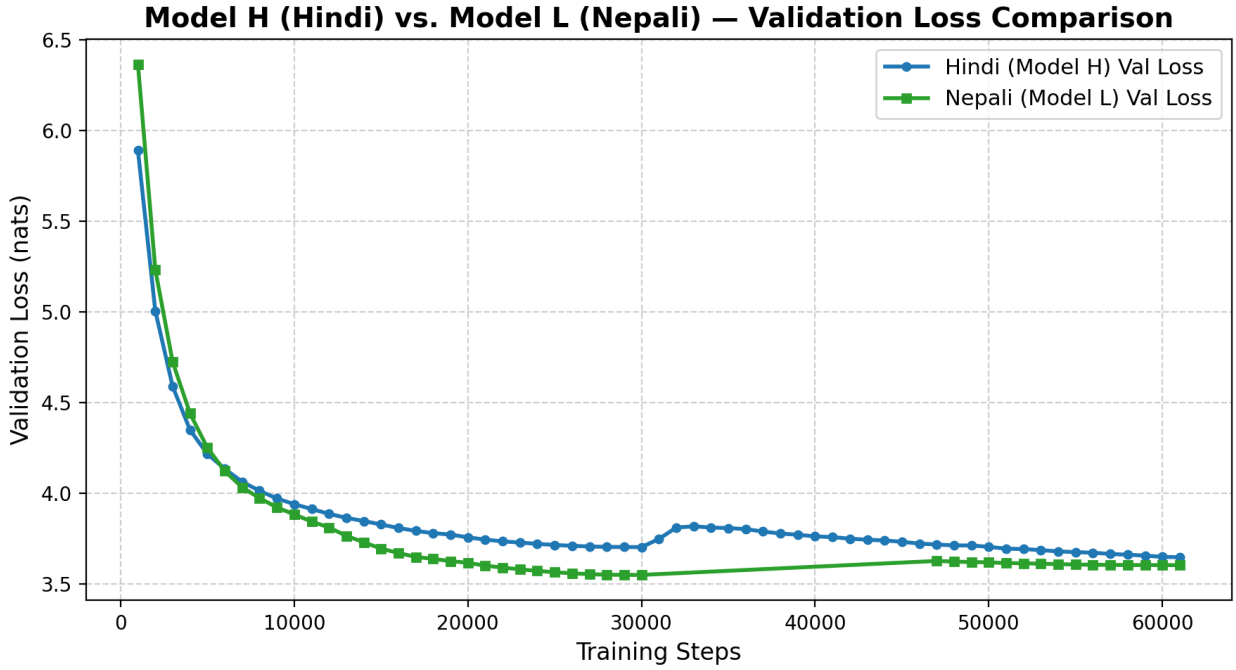


Figure 1: Validation loss trajectory during 61,036 pretraining steps for Model H and Model L.

easier to model would be a mistake. Nepali tokens average 13.16 bytes compared to 10.31 bytes in Hindi, which compresses cross-entropy when normalized into BPB. Furthermore, the Nepali corpus is derived predominantly from formal news and archival documents, making it much more predictable than Hindi's diverse colloquial and literary corpus.

Generation and Attention Analysis

To test whether Model L's low loss hid degenerate behavior, I generated continuations using greedy decoding and temperature sampling, measuring vocabulary diversity via distinct n-grams.

Table 5: Generation diversity and repetition diagnostics across decoding strategies

Decoding Strategy	Model	4-gram repetition	Distinct-1	Distinct-2
Greedy	Hindi (Model H)	0.266	0.427	0.622
Greedy	Nepali (Model L)	0.671	0.229	0.317
Temperature 0.5	Hindi (Model H)	0.101	0.461	0.734
Temperature 0.5	Nepali (Model L)	0.335	0.398	0.628

Under greedy decoding, Nepali suffers from repeating the same phrases, repeating 4-grams 67.1% of the time. Hindi, by contrast, generates far more diverse text (26.6% repetition). This confirms that Model L's low pretraining loss was a consequence of learning a narrower, repetitive news distribution rather than acquiring deeper linguistic generalization. Temperature sampling ($T = 0.5$) successfully broke the repetitive cycles in both models.

I also conducted an attention analysis on the Phase 2 base models to see how they process text. Looking closely at the attention weights showed that early layers (like Layer 0) focus heavily on immediate neighbors (concentrated diagonal attention). By the deeper layers (like Layer 5), the attention becomes much broader with a longer average attention distance, allowing the model to capture larger sentence-level semantic dependencies. I also verified causal masking fidelity: modifying the final input token produced strictly zero perturbation in prior hidden states. Having confirmed that both base models successfully internalized language syntax, I proceeded to Phase 3 to test structured deductive reasoning.

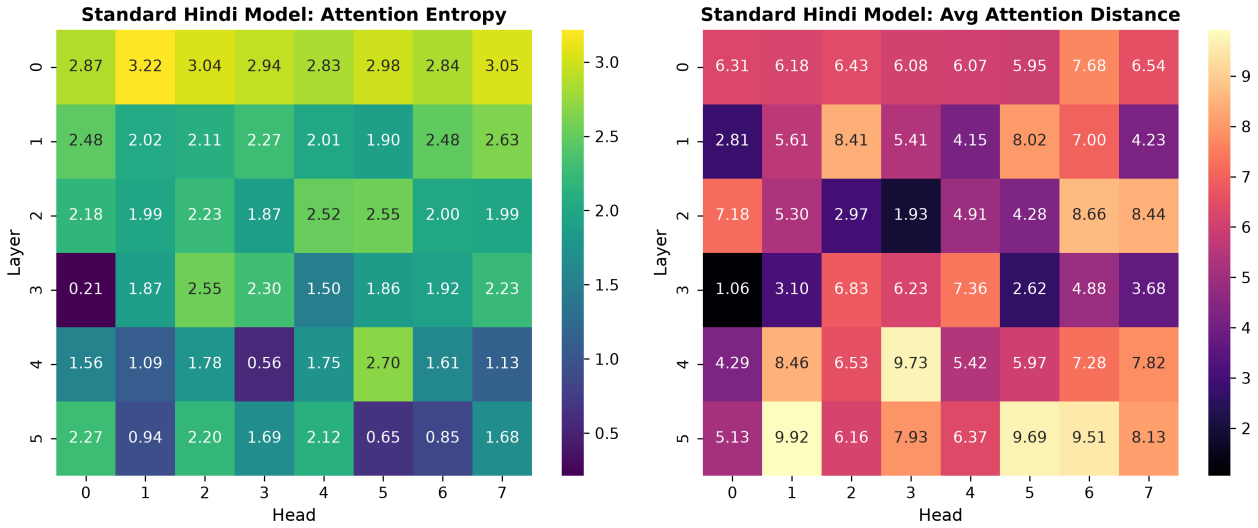


Figure 2: Hindi attention entropy and mean attention distance across layers and heads. Early layers show low entropy, late layers show higher entropy and longer semantic context windows.

Phase 3: Reasoning Finetuning

Synthetic Reasoning Dataset

While the Phase 2 base models generate sentences that follow basic grammar rules, open-ended language models are prone to hallucination. In Phase 3, I investigated whether these 24M-parameter models could acquire systematic reasoning capabilities through supervised instruction finetuning. I built a script that generates complex relational and logical problems across both languages, ensuring that the Nepali generator was implemented with native morphology rather than mechanical translations of Hindi templates.

Table 6: Reasoning dataset structure and task complexity tiers

Tier	Task Focus and Problem Structure
Tier 1	Direct numeric and attribute comparisons between two entities.
Tier 2	Transitive chains (A ¿ B ¿ C), ranking, numeric threshold filtering, and difference calculations.
Tier 3	Temporal sequence ordering, logical contradiction checks, rule classification, and conditional deduction.

The dataset encompasses 13 distinct problem categories totaling 9,000 validated examples per language (split 70/15/15 into train/validation/test). I generated this data entirely from scratch using a Python script, combining 26 real-world attributes (like speed in km/h or price in rupees) with 53 different phrasing templates. Each training example was formatted as a prompt followed by `\n Answer: answer`. To prevent data leakage, I used a fixed seed (`_SPLIT_SEED = 20260913`) to partition the entity pools before generating the text, meaning the models were tested on entities and phrasings they had genuinely never seen before.

Finetuning Protocol and Loss

I adapted each model by resuming training from its Phase 2 pretrained checkpoint while keeping the tokenizer frozen. I implemented selective loss masking by assigning a target label of -100 to all prompt tokens, ensuring that gradient updates backpropagate strictly through the generated answer tokens. Training proceeded for 3 epochs with AdamW (5×10^{-5}, cosine decay).

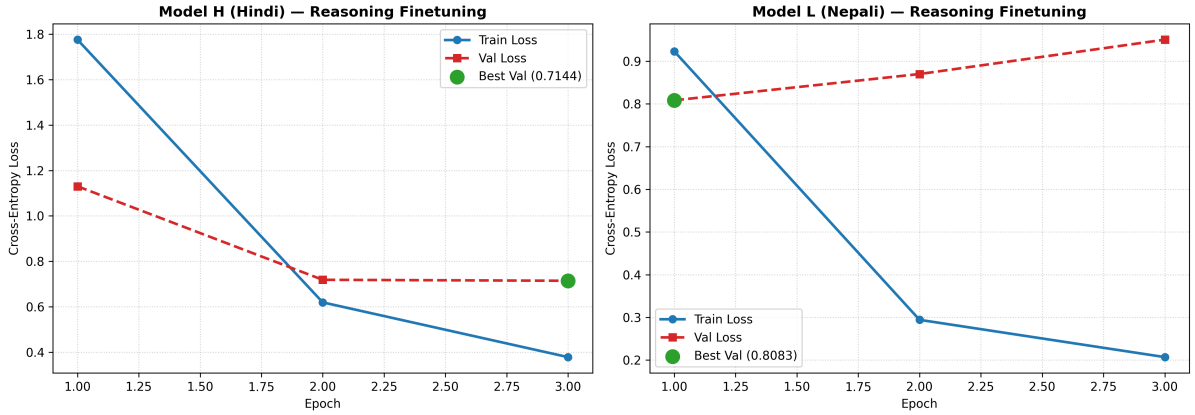


Figure 3: Finetuning loss curves. Hindi generalizes steadily while Nepali overfits after Epoch 1.

The resulting loss trajectories revealed a dramatic contrast between the two language pipelines. Hindi (Model H) displayed very stable training, so I selected its epoch 3 checkpoint. Nepali (Model L) began with a lower initial loss but quickly overfit, with validation loss hitting its minimum at epoch 1 and worsening thereafter. I selected its epoch 1 checkpoint. This overfitting is driven by a major difference in how the tokenizers work: Model L used a character-level BPE tokenizer, which fragments Nepali words into smaller, less semantically cohesive units compared to Hindi's byte-level BPE. Combined with a narrower pretraining corpus, Model L's base representations were not as well-developed, causing it to memorize

synthetic templates rapidly rather than learning generalized logic.

Reasoning Results and Analysis

I evaluated the 1,350 held-out test prompts under greedy decoding. **Strict Exact Match** requires the entire generated text to match the gold target string identically, while **Substring Match** extracts the first generated sentence and verifies whether the gold entity is present.

Table 7: Held-out reasoning accuracy across pretrained and finetuned models (percent)

Evaluation Metric	Hindi (Model H)	Nepali (Model L)
Pretrained strict exact match	0.00%	0.00%
Finetuned strict exact match	0.00%	0.00%
Pretrained substring match	1.04%	1.93%
Finetuned substring match	27.26%	26.67%
Tier 1 after finetuning	38.98%	37.50%
Tier 2 after finetuning	18.50%	17.45%
Tier 3 after finetuning	31.95%	34.49%

Strict exact match is 0.00% because target labels are short entity names, whereas the finetuned models learned to produce full, natural explanatory sentences. Because greedy decoding continues until the end-of-sequence token, full string matching is overly punitive. Substring matching provides a clearer picture:

- **Tier 1 (Pairwise Comparison):** The prompt mentions two candidate entities (a 50% baseline). Model H achieves 38.98% and Model L achieves 37.50%, both falling below the naive 50% copy baseline.
- **Tier 2 (Multi-Step Logic):** With 3 to 4 candidates in ranking tasks, the copy baseline is 25-33%. The models score 18.50% and 17.45%, indicating that relational chains remain largely unlearned.
- **Tier 3 (Rule Classification):** Scores rise to ∼32-34%, driven by binary yes/no questions which inherently carry a 50% prior.

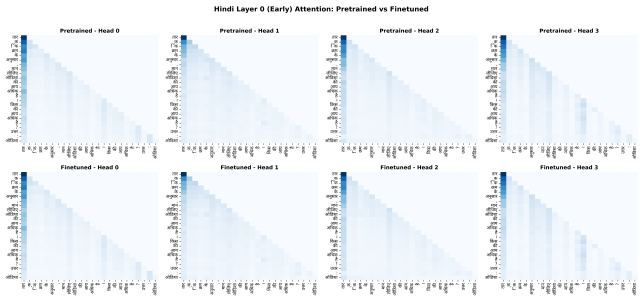
When inspecting the outputs, I noticed three common failure modes:

1. **No stopping:** The model often correctly generated the entity but then kept writing a full sentence instead of stopping, which is why the strict exact match score is 0%.
2. **Position bias:** On Tier 1 questions, it disproportionately guessed the entity that was mentioned last in the prompt, leading to scores below random chance.
3. **Chain collapse:** For 4-hop chain questions, it frequently guessed an entity from the middle of the chain rather than the extreme ends.

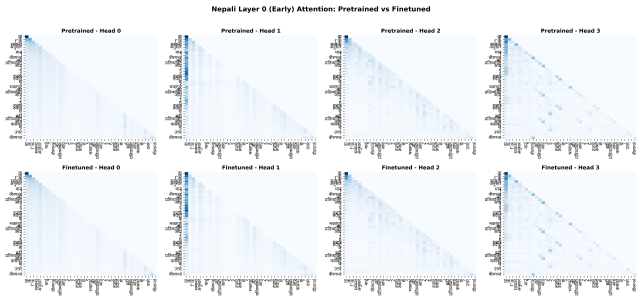
Ultimately, finetuning taught the models how to format an answer and copy an entity, but it did not teach them actual logical reasoning.

Looking at the attention maps before and after finetuning explains why this happens. The early layers (like Layer 0) barely changed, they kept their strong focus on immediate neighbors and the first token (an attention sink). However, the late layers changed significantly. For example, in Hindi Layer 5, Head 1 developed a sharp link between question words (like 'kis ki' [whose]) and the attribute noun (like 'aay' [income]), while Head 2 strengthened its focus on the target entity (like 'Odisha') to copy it into the answer. Nepali showed a similar pattern. Both models learned the format of where to copy the answer from, but lacked the capacity and data to actually learn the math or logic needed to pick the right entity.

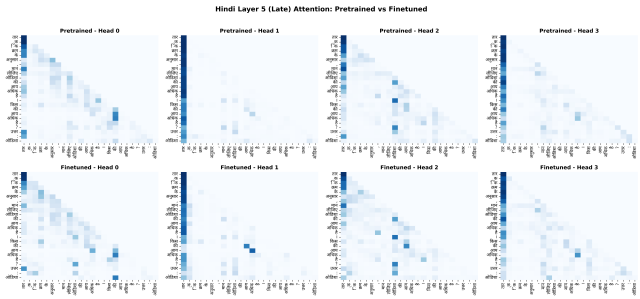
Final Project Comparison 1. Data scale and quality: Hindi had an abundant multi-domain crawl. Nepali relied on a handful of news sites, meaning Model L's corpus was far narrower and more



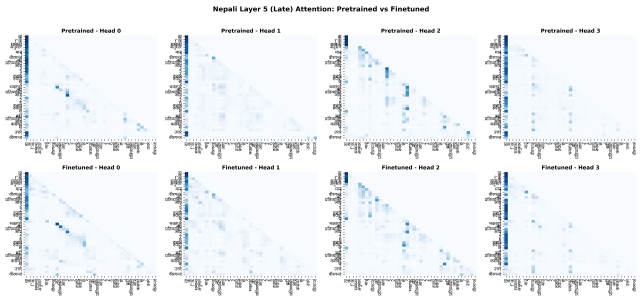
(a) Hindi Layer 0 (Early)



(b) Nepali Layer 0 (Early)



(c) Hindi Layer 5 (Late)



(d) Nepali Layer 5 (Late)

Figure 4: Attention maps before and after finetuning. Late layers develop sharp retrieval and copy circuits.

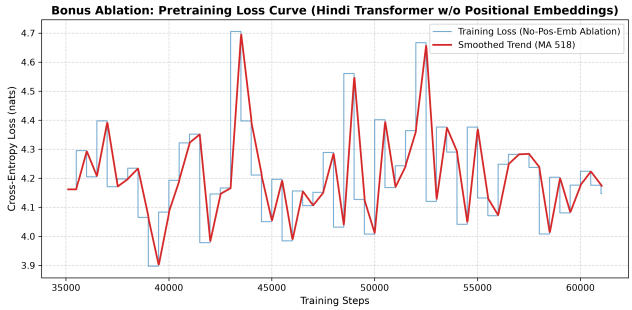
repetitive at the same token count. **2. LM vs Reasoning results:** Nepali achieved better LM metrics (lower BPB), but this just reflects its repetitive, easier-to-predict corpus grading its own homework. On reasoning, they scored identically ($\sim$27%) because the synthetic datasets were perfectly size-matched. **3. Tokenizer factors:** The Nepali BPE learned longer tokens because of the repetitive corpus. This flatters BPB but fragments unseen test entities badly. Corpus diversity mattered more than the tokenizer itself. **4. Evidence for differences:** The loss curves (Nepali overfits in 1 epoch), matched reasoning scores, and identical late-layer attention maps prove the differences stem entirely from the underlying data distributions and not the model architecture.

Bonus: Positional Embedding Ablation

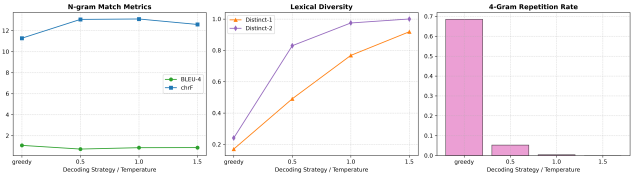
We observed how attention heads rely on sequential structure to build syntactic and copy circuits. This raises a fundamental architectural question: Does causal masking alone supply sufficient sequential ordering, or are learned positional embeddings strictly absolutely necessary? To answer this, I conducted a full ablation study, retraining Model H from scratch for 61,036 steps with identical hyperparameters and compute budget, but with learned positional embeddings completely removed.

Table 8: Standard Hindi Model H versus the No-Position Ablation model

Evaluation Metric	Standard Model H	No-Position Ablation	Relative Change
Validation loss (nats)	3.6487	4.0697	+11.5%
Perplexity (PPL)	39.11	58.54	+49.7%
Bits per byte (BPB)	0.5131	0.5674	+10.6%
4-gram repetition rate	26.59%	68.52%	+41.93 points
Distinct-1 vocabulary diversity	0.4270	0.1703	-60.1% diversity
Mean attention distance	7.15 tokens	2.18 tokens	Collapsed to local



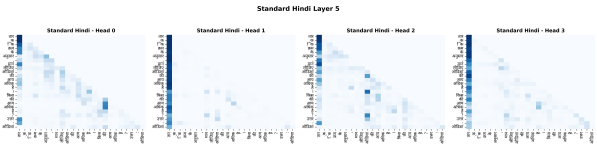
(a) Training Loss



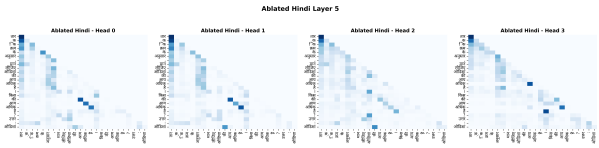
(b) Generation Diversity

Figure 5: The ablated model plateaus at a higher loss and generates highly repetitive text.

The results demonstrate that positional embeddings are essential. Without positional inductive bias, the model suffers a 49.7% spike in perplexity (39.11 to 58.54) and a severe collapse in generative diversity: 4-gram repetition skyrockets from 26.59% to 68.52%, while Distinct-1 drops from 0.4270 to 0.1703. While causal masking breaks the lack of sequence order by enforcing a lower-triangular visibility matrix, it cannot convey relative token distances, causing the Transformer to fail to understand sentence structure.



(a) Standard Hindi: Layer 5



(b) Ablated Hindi: Layer 5

Figure 6: Attention patterns. The ablated model collapses into a self-attending diagonal identity mapping.

The standard model exhibits distinct layer-wise functional specialization, with low-entropy local attention in early layers and higher-entropy long-range attention in later layers. The ablated model is completely flat across all layers. Mean attention distance collapses from 7.15 down to 2.18. This proves that the attention mechanism cannot organize itself without position information to anchor heads to, confirming that positional embeddings are absolutely necessary for Transformer language modeling.

Hypothesis vs. Findings. The theoretical expectation was that self-attention, being essentially a set-operation, would fail completely to learn syntax without positional embeddings. However, I hypothesized that causal masking which inherently provides the model with a basic sense of "past" versus "future" might provide just enough sequential bias for the model to generate somewhat coherent short phrases.

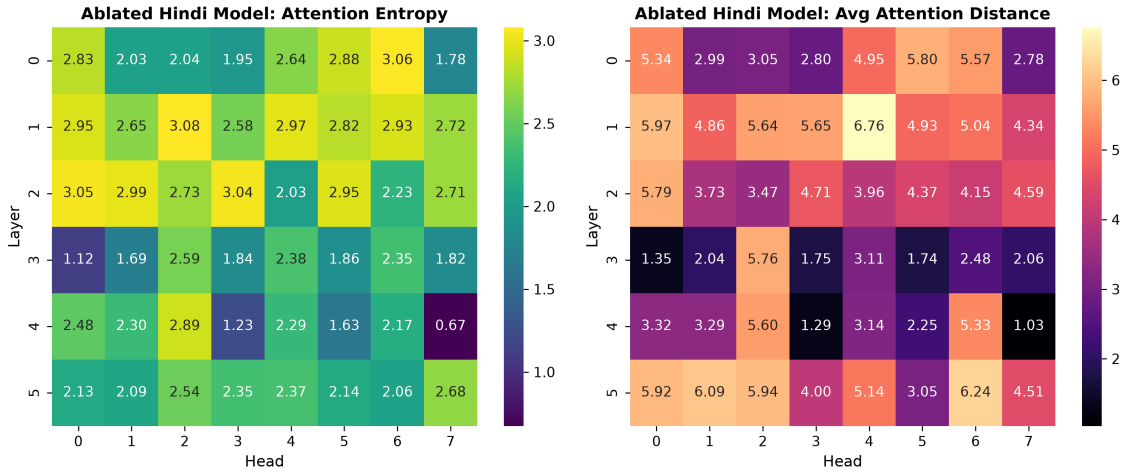


Figure 7: Ablated model attention entropy and distance. The metrics are flat across all layers.

Our findings ultimately reject this hypothesis and strongly confirm the theoretical expectation. While causal masking breaks perfect sequence invariance, the severe spike in perplexity (from 39.11 to 58.54) and the dramatic collapse in vocabulary diversity prove it is not enough to build long-range syntactic dependencies. As shown by the attention distance dropping to 2.18 tokens, the ablated model devolved into looking only at its immediate neighbors, functioning like a basic local bag-of-words. It completely lost the layer-wise specialization seen in Phase 2 (where deeper layers look further back in the sentence). This confirms that while causal masking creates a one-way street for information flow, explicit positional embeddings are absolutely required for the attention mechanism to learn structure and syntax.